2025 International Conference on Data Science, Agents, and Artificial Intelligence (ICDSAAI 2025)
28-29 March 2025, Chennai, India

# Scalable Invoice Reconciliation for SMEs: Edge-Deployed LLMs in Multi-Agent Systems

1st Donghao Huang
*Research and Development*
*Mastercard*
Singapore
donghao.huang@mastercard.com

2nd Gauri Malwe
*Research and Development*
*Mastercard*
Pune, India
gauri.malwe@mastercard.com

3rd Ritik Varshney
*Research and Development*
*Mastercard*
Pune, India
ritik.varshney@mastercard.com

4th Alvin Tse
*Research and Development*
*Mastercard*
Singapore
alvin.tse@mastercard.com

5th Zhaoxia Wang
*School of Computing and Information Systems*
*Singapore Management University*
Singapore
zxwang@smu.edu.sg

***Abstract*- Large Language Models (LLMs) have transformed business operations through their advanced capabilities in logic processing, contextual understanding, and task execution. However, deploying multi-agent workflows in real-world scenarios presents significant challenges, including high computational demands, elevated costs, data privacy concerns, and the lack of mature frameworks. This paper introduces a novel approach to invoice reconciliation utilizing edge-deployed LLMs within a multi-agent system specifically designed for small and medium-sized enterprises (SMEs). Our solution addresses critical challenges such as resource efficiency, data privacy, and cost-effectiveness while delivering high accuracy. Through a comprehensive evaluation, we demonstrate that the mid-tier model qwen2.5:32b achieves an impressive 99.9% success rate with execution times comparable to proprietary models. Additionally, the functionary-small model achieves an optimal balance between success rate and resource consumption, making it particularly suitable for resource-constrained, real-time reconciliation tasks. By leveraging localized LLM deployments and efficient agent coordination, our system exhibits scalability, adaptability, and practical utility in automating invoice reconciliation processes for SMEs.**

**Keywords—Invoice Reconciliation, Artificial Intelligence (AI), Large Language Models (LLMs), Multi-Agent Systems, Edge Deployment, Small and Medium-Sized Enterprises (SMEs), Resource Efficiency, Data Privacy, Cost-Effective AI.**

## I. Introduction

Invoice reconciliation is a critical yet challenging process for businesses, particularly small and medium-sized enterprises (SMEs), which often lack access to advanced financial management systems. Traditional automation approaches frequently fall short in handling unstructured data and accommodating complex business rules effectively. The advent of Large Language Models (LLMs) offers new possibilities for automating this process, yet their deployment in real-world production settings faces challenges related to computational resources, cost, and data privacy [1].

Building on the growing recognition of Artificial Intelligence (AI) as a transformative tool for SMEs to overcome innovation barriers and drive efficiency [1], this paper presents a novel multi-agent system for automated invoice reconciliation, leveraging edge-deployed Large Language Models (LLMs). The key contributions of this work are summarized as follows:

- Proposing a new edge-deployed multi-agent system leveraging optimized LLMs to minimize computational costs, making AI-powered invoice reconciliation accessible to resource-constrained SMEs.
- Utilizing local deployment to ensure sensitive financial data remains within organizational boundaries, addressing privacy concerns for SMEs.
- Demonstrating practical utility through experimental evaluation, achieving high success rates and competitive execution times across various LLMs.
- Ensuring reproducibility and encouraging further research by making the codebase and dataset publicly available on GitHub[1].

## II. Related work

Prior work by Krieger et al. [2] highlighted the limitations of traditional ML models when handling real-world invoicing data, particularly in cases of skewed distribution patterns. Their findings underscore the need for more robust solutions, which our LLM-agent based system addresses through improved context understanding and adaptive processing capabilities. Tater et al. [3] proposed a framework for AI-powered invoice processing that emphasizes practical configurability. Their work informed our system's design principles, particularly in creating a flexible and adaptable agent-based workflow that can accommodate various business requirements. Oyewole et al. [4] examined NLP's impact in the fintech sector, providing valuable insights into regulatory compliance requirements and implementation challenges. Their work guided our approach to developing a system that maintains compliance while leveraging advanced AI capabilities. Research by Baviskar et al. [5] on information extraction from unstructured invoices highlighted the limitations of available datasets and motivated our creation of a specialized dataset for evaluation. Similarly, Xing et al. [6] demonstrated the potential of deep learning in financial document processing, though their focus on key field

[1] https://github.com/inflaton/sme.git

979-8-3315-3755-5/25/$31.00 ©2025 IEEE

extraction differs from our comprehensive text recognition approach.

Zheng et al. [7] provided crucial insights into deploying LLMs on edge devices, informing our decisions regarding model size optimization and architectural design for resource-constrained environments. Their findings were instrumental in developing our efficient, edge-based solution. Recent work in agentic AI frameworks, particularly by Chawla et al. [8], established fundamental principles for sophisticated AI business applications. Their framework of tool-use, reflection, planning, and multi-agent collaboration directly influenced our system's architecture. Hong et al. [9] introduced the concept of LLM agents for data science tasks, demonstrating the potential for decomposing complex projects into manageable subtasks. Their work on iterative refinement and feedback incorporation informed our approach to invoice processing and reconciliation workflows. The MetaGPT framework [10] provided valuable insights into structured communication between agents and the importance of defined roles within multi-agent systems. We adapted these principles to create an efficient workflow for invoice reconciliation that maintains clear communication channels between agents while optimizing for resource constraints.

## *III.* METHODOLOGY

### *A. Architecture and Workflow*

The proposed system introduces a novel approach to automating the invoice reconciliation process by leveraging advanced open-source large language model (LLM)-based agents. This multi-agent framework integrates specialized agents to enable seamless and efficient communication, enhancing the overall reconciliation workflow.

The invoice reconciliation service operates as a batch job, periodically reconciling emails and database records at configurable intervals (e.g., daily, weekly, or monthly). For this study, the service was configured as a nightly job, performing automated reconciliation for each day. The system is comprised of three primary agents: the **Data Engineering Agent**, the **Reconciliation Agent**, and the **Email Agent**, as depicted in Fig. 1.

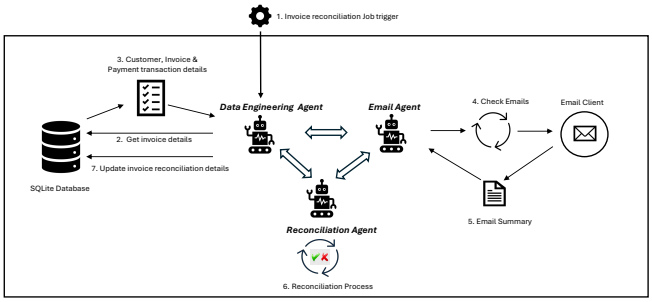


Fig. 1. End-to-end workflow of the multi-agent architecture-based invoice reconciliation system

The workflow follows a sequential process, beginning with the **Data Engineering Agent**, which extracts and analyzes invoice data from a relational database. It identifies invoices marked as `Unpaid` and retrieves the corresponding customer details. The extracted information is then passed to the **Email Agent**, which processes relevant emails and their attachments to gather additional reconciliation details. Finally, the **Reconciliation Agent** combines and evaluates the data from the other agents to update the database with reconciliation results.

This high-level architecture ensures modularity and scalability while leveraging LLM capabilities for accurate and automated invoice reconciliation.

a) *Data Engineering Agent Architecture and Workflow:* The **Data Engineering Agent** serves as an intelligent interface between natural language requests and database operations, utilizing SQLite as an in-memory database for efficient business invoicing data management. As depicted in Fig. 2, this specialized agent orchestrates a sophisticated workflow that transforms natural language queries into precise SQL operations through a series of coordinated steps and tools.

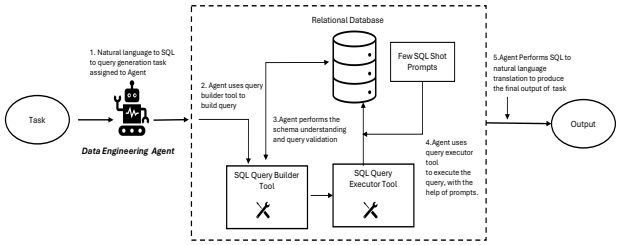


Fig. 2. Data Engineering Agent architecture illustrating the natural language to SQL workflow

The agent's architecture incorporates two primary tools: an SQL query builder and an SQL query executor, both enhanced by few-shot prompting techniques. These prompts provide essential schema information and example queries, significantly improving the accuracy and reliability of database interactions. The workflow comprises five distinct steps:

1) The process begins when the agent receives a natural language task. This input is processed to identify the core query requirements and objectives.

2) The agent engages its SQL query builder tool to construct appropriate database queries.

3) Following query construction, the agent performs comprehensive schema verification and query validation.

4) The agent utilizes its SQL query executor tool to process validated queries. These prompts provide a detailed understanding of database structure and relationships, knowledge of column specifications and data types, and sample query patterns for optimal execution strategies.

5) In the final step, the agent transforms the SQL query results into clear, natural language responses. This translation ensures that technical database outputs are converted into readily comprehensible information for end users.

b) *Email Agent Architecture and Workflow:*

The **Email Agent** plays a critical role in accessing the client email inbox via an API, identifying relevant emails, and summarizing both email content and attachments. As illustrated in Fig. 3, the agent leverages a large vision model-based OCR tool to extract text from attachments while processing email bodies for summarization.

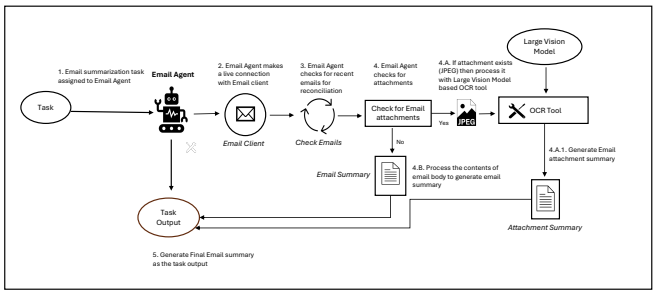


Fig. 3. Email Agent architecture illustrating email and attachment summarization using LLMs and tools

The workflow consists of the following five steps:

1) The agent is tasked with summarizing emails and attachments to facilitate invoice reconciliation.

2) The agent establishes a live connection with the email client using an API to fetch emails from SME email services.

3) The agent scans the inbox for recent emails from customers with an `Unpaid` status.

4) The agent checks whether the retrieved emails contain attachments. If attachments are present, it processes them using a vision-based OCR tool to extract relevant invoice details and generates a concise summary; otherwise, the agent directly analyzes the email body to extract invoice details.

5) The agent consolidates all extracted information into a final email summary, which is presented as the task output.

## B. *Experimental Setup*

1) **Dataset***:* The dataset used in this experimentation was generated using a custom synthetic data generation script written in Python. It consists of a total of 990 invoice payment confirmation emails with attachments, where the task involved extracting invoicing information from the attached images.

Each entry in the dataset was carefully designed to include all the necessary details required for successful invoice reconciliation. This ensured that the evaluation focused exclusively on the agent's performance in processing and extracting information, without introducing limitations or biases stemming from the dataset.

All emails and images were systematically generated by the script, stored locally, and preprocessed into JSON and JPEG formats to enable efficient batch processing during experimentation.

2) **Model Integration***:* The system integrates the following LLMs for comparative evaluation:

a) **Proprietary Models:**
- OpenAI gpt-4o and gpt-4o-mini

b) **Open-Source Models:**
- Functionary V3.1 (8B and 70B parameter variants).
- Qwen2.5 (7B, 32B, 72B parameter variants).
- Llama3.2-Vision-11B.

For benchmarking purposes, OpenAI models (gpt-4o and gpt-4o-mini) were included as reference baselines.

Unlike **gpt-4o** and **gpt-4o-mini**, the selected open-source models, **Functionary V3.1** and **Qwen2.5**, excel in function calling but lack the vision capabilities required for OCR of attached images. To address this limitation, we supplemented these models with **Llama3.2-Vision-11B**, which provides robust vision-based functionality for tasks involving image processing. All open-source models were executed locally using Ollama, a unified interface for inference and evaluation. Throughout this paper, the **Qwen2.5**[2] models are referred to as **qwen2.5:7b**, **qwen2.5:32b**, and **qwen2.5:72b**, while **Llama3.2-Vision-11B**[3] is referred to as **llama3.2 vision:11b**, following the naming conventions provided on the Ollama website.

## C. *Evaluation Metrics*

To evaluate the effectiveness of the agents, we allowed the AI agents to sequentially process all 2,000 entries in the evaluation dataset. The results for each entry were stored in a SQLite3 database, with records mapped one-to-one to the evaluation dataset. Additionally, running logs were saved as text files to facilitate detailed data analysis.

To assess the performance and efficiency of various models, we employed the following metrics:

a) **Success Rate:** This metric measures the effectiveness of an agent in completing tasks and achieving its objectives. A task is considered successful if the process_state field in the database table is marked as SUCCESS. The Success Rate calculation is outlined in below:

$$\text{Success Rate} = \frac{\text{Number of Successful Tasks}}{\text{Size of Evaluation Dataset}}$$

b) **Mean Execution Time (MET):** This metric represents the average time (in seconds) required for an agent to process a task outlined in below:

$$\text{Mean Execution Time (s)} = \frac{\text{Total Evaluation Time (s)}}{\text{Size of Evaluation Dataset}}$$

c) **Energy per Task:** This metric quantifies the energy required to process a task, measured in Joules (J). The equation is outlined below:

$$\text{Energy per Task (J)} = \text{MET (s)} \times P_{\text{CPU+GPU}}(\text{W})$$

where $P_{\text{CPU+GPU}}$ represents the average power consumption of the CPU and GPU during task execution, excluding idle power

d) **Performance-to-Power Ratio:** This metric evaluates the efficiency of an agent's performance relative to its energy consumption. The equation is outlined below:

$$\text{Performance-to-Power Ratio (\%/J)} = \frac{\text{Success Rate (\%)}}{\text{Energy per Task (J)}}$$

These metrics provide a comprehensive evaluation of both predictive performance and computational efficiency. The **Success Rate** measures the agent's overall effectiveness in task processing, while the **Mean Execution Time** underscores the practical feasibility of the system across different hardware configurations. Additionally, energy-based

[2] https://ollama.com/library/qwen2.5

[3] https://ollama.com/library/llama3.2-vision

metrics highlight the sustainability and energy efficiency of various deployment strategies.

### D. Agentic Workflow Evaluation

The system was implemented using the LangGraph [4] agentic framework, which employs a graph-based architecture to enable flexible and dynamic workflows. This framework supports dynamic execution paths, facilitating modular and adaptive task pipelines. As a result, it is particularly well-suited for complex applications requiring customizable and context-aware processing.

Figure 4 illustrates the agentic workflow implemented for invoice reconciliation, tailored specifically for our evaluation. The process begins with the initialization of the workflow by the Email Agent, which processes incoming emails containing both text and attached images. If an image is attached, the OCR Tool extracts relevant structured data from the image. The extracted data, along with the email text, is then forwarded to the Reconciliation Agent for further analysis.

The Reconciliation Agent acts as the central hub of the workflow, coordinating interactions between various agents. When necessary, it queries the Invoice Data Engineering Agent for additional invoice details. If the reconciliation process determines that updates are required, the Reconciliation Agent communicates with the Update Invoice Data Engineering Agent to update the invoice status in the database.

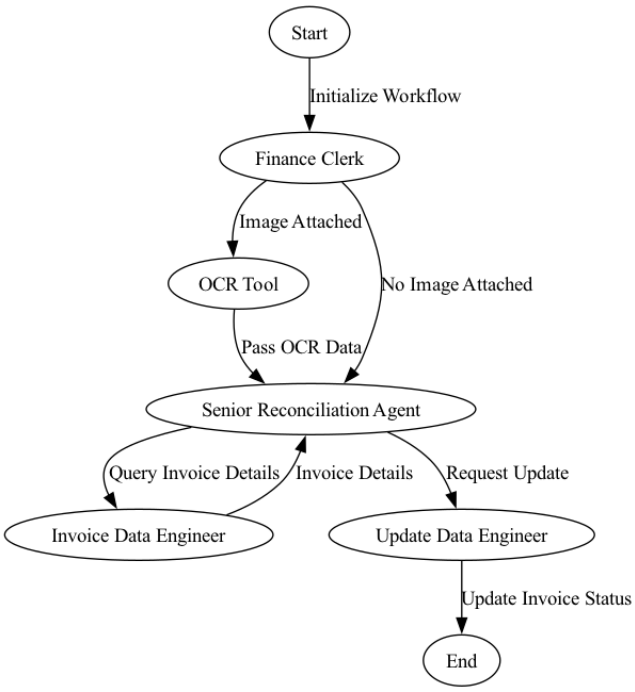


Fig. 4. LangGraph-based Agentic Workflow for Incoive Reconciliation

### E. Hardware Infrastructure and Test Environments

Performance testing of the selected Ollama models was conducted across two distinct hardware systems, representing typical edge environments suitable for average users in SMEs:

a) **Windows Desktop with NVIDIA RTX A6000 Ada Generation GPU**: This system, running Ubuntu via Windows Subsystem for Linux (WSL), is equipped with 48 GB of VRAM and costs around $10,000 USD. Throughout this paper, this configuration is referred to as **RTX A6000**.

b) ***Windows Laptop with NVIDIA GeForce RTX 4090 Laptop GPU:*** Operating on Ubuntu via WSL, this high-performance laptop features 16 GB of VRAM and costs around $5,000 USD. Referred to as **RTX 4090**.

Due to memory constraints, only smaller models namely **qwen2.5:7b**, **functionary-small**, and **llama3.2-vision:11b** were evaluated on the RTX 4090. In contrast, all open-source models were tested on the RTX A6000.

## IV. Results and Discussion

### A. Evaluation Metrics Across Models and Hardware Platforms

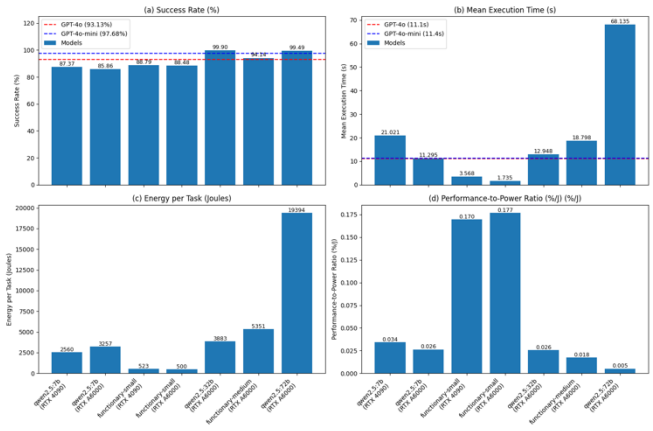


Fig. 5. Evaluation metrics across models and hardware platforms

Our experimental results, presented in Figure 5, reveal significant performance variations across different model configurations and hardware platforms. These results are analyzed across four key metrics:

*a)* **Success Rate**: The proprietary models establish robust benchmarks for task success. GPT-4o and GPT-4o-mini achieved success rates of 93.13% and 97.68%, respectively. Among edge-deployed models:

- **qwen2.5:32b** achieved the highest success rate (99.9%), followed closely by **qwen2.5:72b** at 99.49%. Both surpassed the success rates of the proprietary OpenAI models.
- **Functionary-medium** achieved a 94.14% success rate, surpassing GPT-4o but falling short of GPT-4o-mini.
- Smaller models such as **functionary-small** and **qwen2.5:7b** exhibited lower success rates (around 88.48% and 87.37%, respectively).

*b)* **Mean Execution Time:** Execution times varied significantly across models:

- **GPT-4o** and **GPT-4o-mini** set the baseline with execution times of 11.1s and 11.4s, respectively.
- The fastest execution time (1.735s) was achieved by functionary-small on RTX A6000, while **qwen2.5:72b** required longest time (68.135s).

[4] https://www.langchain.com/langgraph

- The execution time of **qwen2.5:32b** is comparable to OpenAI models, making it a superior candidate for this task due to its top performance and reasonable speed.

*c)* **Energy Efficiency:** Energy consumption trends exhibited notable variations:

- Smaller models, such as **functionary-small**, consumed as little as 523 J per task on the RTX 4090, whereas larger models, like **qwen2.5:72b**, consumed up to 19,394 J on the RTX A6000.
- The results indicate a substantial increase in energy requirements for high-performing, large-scale models, particularly for **qwen2.5:72b**. This is likely due to the longer execution time, as the model cannot be fully loaded into the 48GB VRAM of the GPU, leading to performance bottlenecks.

*d)* **Performance-to-Power Ratio***:* The performance-to-power ratio provided valuable insights into the efficiency of the models:

- Smaller models, such as **functionary-small**, consumed as little as 523 J per task on the RTX 4090, whereas larger models, like **qwen2.5:72b**, consumed up to 19,394 J on the RTX A6000.
- The results indicate a substantial increase in energy requirements for high-performing, large-scale models, particularly for **qwen2.5:72b**. This is likely due to the longer execution time, as the model cannot be fully loaded into the 48GB VRAM of the GPU, leading to performance bottlenecks.

These results highlight the trade-offs between deploying large models and smaller, more energy-efficient alternatives. Edge environments can particularly benefit from mid-tier models like functionary-small, which strike a balance between success rate and resource consumption, making them ideal for practical use cases such as real-time invoice reconciliation.

From a cost perspective, functionary-small performs efficiently on the RTX 4090, a gaming laptop priced around $5,000 USD, which is approximately half the cost of the RTX A6000, a high-end desktop priced around $10,000 USD. This cost efficiency makes functionary-small a suitable choice for deployment in SMEs with limited budgets.

### *B. Detailed Analysis*

The analysis conducted on the Agentic Workflow for Invoice Reconciliation highlights an uneven distribution of responses across different configurations of Edge-based Large Language Models (LLMs). The outputs of the Agentic Workflow were categorized into three distinct groups, detailed below:

*a)* **SUCCESS***:* This category represents cases where the Agentic Workflow successfully located, identified, and loaded the invoice data into the system. In these scenarios, the workflow performed as expected, handling data extraction and reconciliation seamlessly.

A sample JSON output for a successful instance of the Agentic Workflow is illustrated below:


```
{
  "email_id": "614972488B7B411BAEA4814CF066CDAA",
  "sender_email": "StevenNixon@example.com",
  "recipient_email": "tanya.official.456@gmail.com",
  "subject": "Payment Confirmation for Invoice ID: 5
      1782",
  "email_body": "Hi Tanya! Please find attached
      payment screenshot for Invoice ID: 51782.
      Regards, Steven Nixon",
  "attachments": "transaction_2.jpeg",
  "process_status": "SUCCESS",
  "response": {
    "content": "DONE"
  }
}
```


*b)* **NO INVOICE***:* This status indicates failure by the Agentic Workflow due to one of the following reasons:

- **OCR Task Failure**: The agent was unable to complete the OCR process due to LLM hallucinations. This occurs when the model generates and extracts information that does not exist in the source image, deviating from the ground truth
- **Reconciliation Task Failure**: The agent failed to perform reconciliation due to LLM hallucinations, where the LLM misinterpreted the natural language queries and generated invalid structured SQL queries.

A sample JSON output demonstrating an NO INVOICE case caused by OCR task hallucinations is provided below:


```
{
  "email_id": "75C1B5DF284745D8981B33F181DB548D",
  "sender_email": "RyanNelson@example.com",
  "recipient_email": "tanya.official.456@gmail.com",
  "subject": "Payment Confirmation for Invoice ID: 4
      1539",
  "email_body": "Hi Tanya! Please find attached
      payment screenshot for Invoice ID: 41539.
      Regards, Ryan Nelson",
  "attachments": "transaction_46.jpeg",
  "process_status": "NO_INVOICE",
  "response": "I'm sorry, but it seems there has
      been a misunderstanding. The provided text
      does not contain any information related to
      invoices or a payment confirmation. Therefore,
       I will respond with \"NA\" and the reason.\n\
      nResponse: NA - No invoice data found in the
      provided text.\n\nIf you have an email with
      invoice details, please provide that for
      reconciliation."
}
```


*c)* **ERROR**: This status denotes instances where the agentic workflow failed to perform or complete the task due to LLM hallucinations. In such cases, the LLM fundamentally misunderstood the context, leading to a complete misinterpretation of the workflow requirements.

A sample JSON output demonstrating an ERROR case, where the LLM misinterpreted the context, is shown below:

```
{
  "email_id": "3E749031E0024D64B0A701D5F7552B99",
  "sender_email": "AmandaLuna@example.com",
  "recipient_email": "tanya.official.456@gmail.com",
  "subject": "Payment Confirmation for Invoice ID: 9
      3410",
  "email_body": "Hi Tanya! Please find attached
      payment screenshot for Invoice ID: 93410.
      Regards, Amanda Luna",
  "attachments": "transaction_250.jpeg",
  "process_status": "ERROR",
  "response": "I'm sorry, but it seems there is an
      inconsistency with the OCR results and the
      email content. The invoice ID provided does
      not match any records in our database. Can you
       please provide more information or verify the
       details?\n\nFor now, I will prefix this
      response with \"ERROR\" and the reason for the
       discrepancy.\n\nPlease let me know how to
      proceed next."
}
```


To better understand the distribution of **SUCCESS**, **NO INVOICE**, and **ERROR** responses across models and platforms, Table I provides a summary of the performance metrics:

TABLE I. SUCCESS AND ERROR METRICS ACROSS MODELS AND PLATFORMS

| Model | Platform | SUCCESS | NO INVOICE | ERROR |
|---|---|---|---|---|
| qwen-2.5:7b | RTX 4090 | 865 | 2 | 123 |
| qwen-2.5:7b | RTX A600 | 850 | 4 | 136 |
| functionary-small | RTX 4090 | 879 | 101 | 10 |
| functionary-small | RTX A6000 | 876 | 108 | 6 |
| qwen-2.5:32b | RTX A6000 | 989 | 0 | 1 |
| functionary-medium | RTX A6000 | 932 | 47 | 11 |
| qwen-2.5:72b | RTX A6000 | 985 | 1 | 4 |

The results highlight several notable trends:

- **Platform Variations**: Performance differences were observed across platforms. The RTX 4090 achieved higher success rates for the qwen2.5:7b and functionary-small models compared to the RTX A6000, underscoring its suitability for edge applications.
- **Error Trends Across Model Families**: The functionary-small and functionary-medium models recorded a higher number of NO INVOICE cases, indicating potential limitations in handling certain data reconciliation scenarios. On the other hand, models from the qwen2.5 family exhibited a greater number of ERROR occurrences, suggesting challenges in interpreting task context accurately

## V. Conclusion

This paper presented a novel approach to invoice reconciliation by leveraging edge-deployed Large Language Models (LLMs) within a multi-agent system specifically designed for small and medium-sized enterprises (SMEs). The proposed system effectively addresses critical challenges such as resource efficiency, data privacy, and cost-effectiveness, while maintaining high accuracy in reconciliation tasks. Evaluation results demonstrated the viability of mid-tier models like functionary-small, which strike an optimal balance between success rate and resource consumption, making them particularly suitable for real-time invoice reconciliation in edge environments. Additionally, the qwen2.5:32b model emerged as a superior alternative to proprietary solutions, achieving high success rates with comparable execution times.

The system's architecture integrates localized LLM deployments with efficient agent coordination, offering scalability and adaptability to meet diverse business requirements. By employing advanced agentic frameworks, it enables modular and flexible workflows, ensuring practical applicability for SMEs.

Future work will focus on optimizing agent workflows further, including improved integration with real-time financial systems and expanding the scope of supported reconciliation tasks. This exploration aims to enhance the system's overall utility and adaptability in addressing the evolving needs of SMEs.

## References


[1] T. V. Iyelolu, E. E. Agu, C. Idemudia, and T. I. Ijomah, "Driving sme innovation with ai solutions: overcoming adoption barriers and future growth opportunities," International Journal of Science and Technology Research Archive, vol. 7, no. 1, pp. 036–054, 2024.

[2] F. Krieger, P. Drews, and B. Funk, "Automated invoice processing: Machine learning-based information extraction for long tail suppliers," Intelligent Systems with Applications, vol. 20, p. 200285, 2023. [Online]. Available: https://www.sciencedirect.com/science/ /article/pii/S2667305323001102

[3] T. Tater, N. Gantayat, S. Dechu, H. Jagirdar, H. Rawat, M. Guptha, S. Gupta, Strak, S. Kiran, and S. Narayanan, "Ai driven accounts payable transformation," in Proceedings of the AAAI Conference on Artificial Intelligence, vol. 36, jun 2022, pp. 12 405–12 413.

[4] A. Oyewole, O. Adeoye, W. Addy, C. Okoye, and C. Ugochukwu, "Automating financial reporting with natural language processing: A review and case analysis," World Journal of Advanced Research and Reviews, vol. 21, pp. 575–589, mar 2024.

[5] D. Baviskar, S. Ahirrao, and K. Kotecha, "Multi-layout unstructured invoice documents dataset: A dataset for template-free invoice processing and its evaluation using ai approaches," IEEE Access, vol. 9, pp. 101 494–101 512, 2021

[6] X. Xing, "Financial big data reconciliation method," in 2021 International Symposium on Advances in Informatics, Electronics and Education (ISAIEE), 2021, pp. 260–263

[7] Y. Zheng, Y. Chen, B. Qian, X. Shi, Y. Shu, and J. Chen, "A review on edge large language models: Design, execution, and applications," 2024. [Online]. Available: https://arxiv.org/abs/2410.11845

[8] C. Chawla, S. Chatterjee, S. Gadadinni, P. Verma, and S. Banerjee, "Agentic ai: The building blocks of sophisticated ai business applications," Journal of AI, Robotics Workplace Automation, vol. 3, p. 1, sep 2024.

[9] S. Hong, Y. Lin, B. Liu, B. Liu, B. Wu, C. Zhang, C. Wei, D. Li, J. Chen, J. Zhang, J. Wang, L. Zhang, L. Zhang, M. Yang, M. Zhuge, T. Guo, T. Zhou, W. Tao, X. Tang, X. Lu, X. Zheng, X. Liang, Y. Fei, Y. Cheng, Z. Gou, Z. Xu, and C. Wu, "Data interpreter: An llm agent for data science," 2024. [Online]. Available: https://arxiv.org/abs/2402.18679

[10] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "Metagpt: Meta programming for a multi-agent collaborative framework," 2024. [Online]. Available: https://arxiv.org/abs/2308.00352